



Récupération Énergétique D'un Ascenseur

Pierrard Antoine
N° de candidat : 46814

Encrage

Causes :

- Fonte Des Glaces
- Réchauffement Climatique
- Explosion Démographique

Conséquences :

- ▷ Diminution de la surface habitable terrestre.
- ▷ Urbanisation croissante
- ▶ Développement vertical des villes

Enjeux :

Soutenir ce développement, en apportant un moyen performant et durable pour se déplacer verticalement.

Près de 70% de la population mondiale vivra en ville d'ici 2050

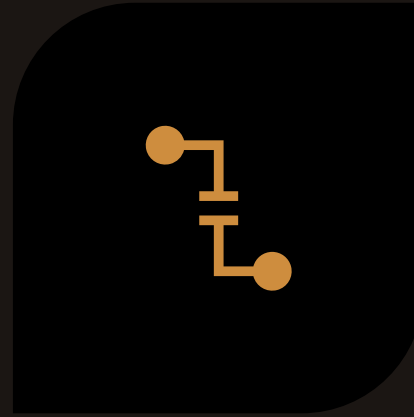
Pourcentage de la population mondiale vivant en zone urbaine



Source : Nation Unies, département des affaires économiques et sociales (2018)



**FREINAGE (AIMANT
PARESSEUX)**



**RÉCUPÉRATION
(CONDENSATEUR CHIMIQUE)**

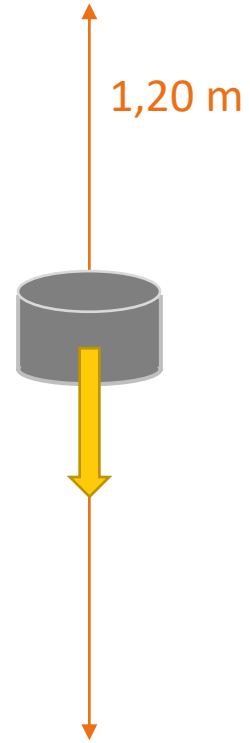


**OPTIMISATION (ALGORITHME
GÉNÉTIQUE)**

Sommaire :



Temps de chute : 5,22 s



Temps de chute : 0,16s

I – Freinage

a : Expérience de

l'aimant paresseux

Tube en cuivre :

$$v_{moy} = 0,23 \text{ m. s}^{-1}$$

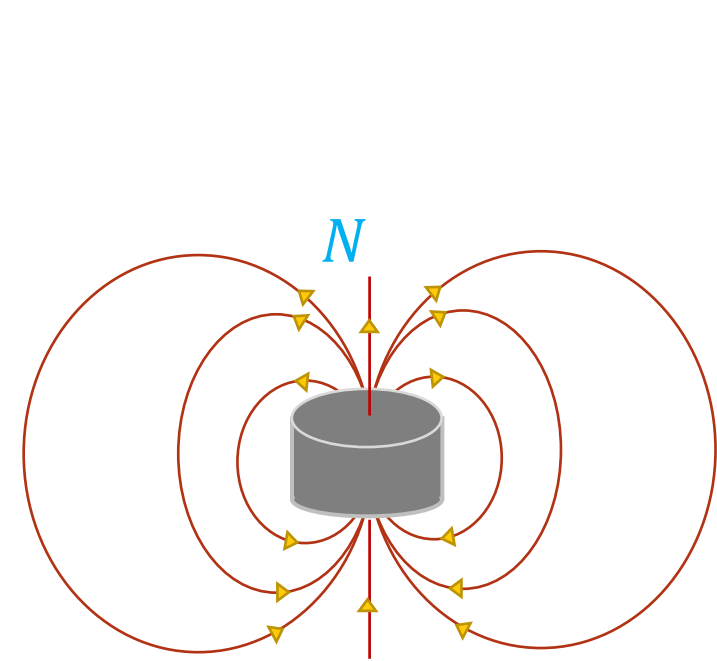
Tube en PVC :

$$v_{moy} = 7,5 \text{ m. s}^{-1}$$

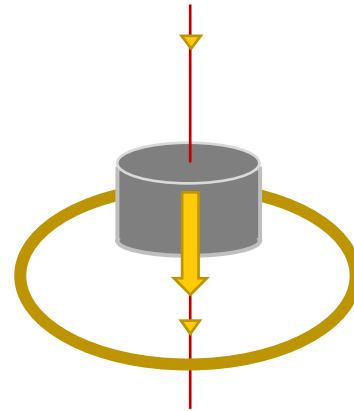
Bilan:

ralentissement : 97%

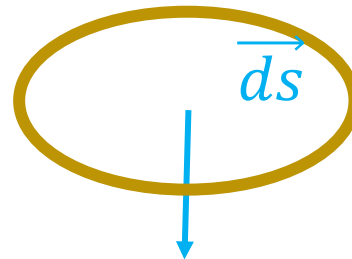
b) Explication qualitative de l'aimant paresseux



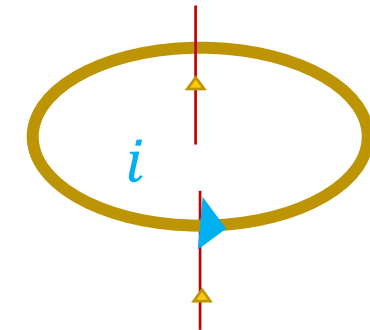
Un aimant est un dipôle magnétique, il génère un champ magnétique



Loi de Lenz – Faraday : $e = -\frac{d\Phi}{dt}$



*Puisque le circuit est fermé, il y a un i induit
Or une spire parcourue par un courant est également
un dipôle magnétique*



*On voit que le champ B s'oppose
à la chute de l'aimant*

Ici tant que l'aimant est au – dessus de la spire :

$$\frac{d\Phi}{dt} > 0 \rightarrow e < 0$$

c) Paramétrage et potentiel vecteur

Potentiel vecteur dû à l'aimant :

$$\vec{A} = \frac{\mu_0 \vec{M} \wedge \vec{u}_r}{4\pi r^2} = \frac{\mu_0 r \vec{M} \wedge \vec{u}_r}{4\pi r^3}$$

Avec $\vec{M}$ le potentiel vecteur :

$$\vec{M} = M \vec{u}_z$$

En projetant $\vec{u}_z$ selon $\vec{u}_\theta$ et $\vec{u}_r$:

$$\vec{u}_z = \cos(\theta) \vec{u}_r - \sin(\theta) \vec{u}_\theta$$

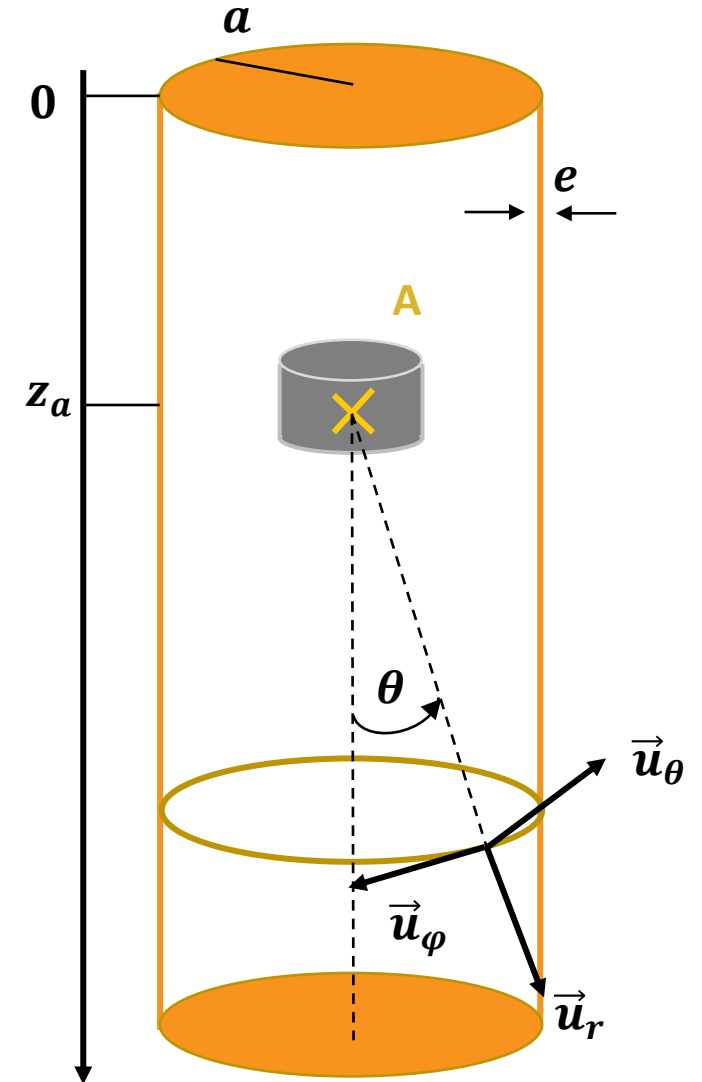
et

$$\vec{A} = \frac{\mu_0 M r}{4\pi r^3} \sin(\theta) \vec{u}_\phi$$

Par des considérations géométriques:

$$r^2 = a^2 + (z - z_A)^2 \text{ et } r \sin(\theta) = a$$

$$\vec{A} = \frac{\mu_0 M a}{4\pi (a^2 + (z - z_A)^2)^{3/2}} \vec{u}_\phi$$



d) Champ Electrostatique au niveau des parois du tube

Par définition :

$$\vec{B} = \overrightarrow{rot}(\vec{A})$$

L'équation de Maxwell – Faraday :

$$\overrightarrow{rot}(\vec{E}) = \frac{-\partial \vec{B}}{\partial t}$$

En injectant :

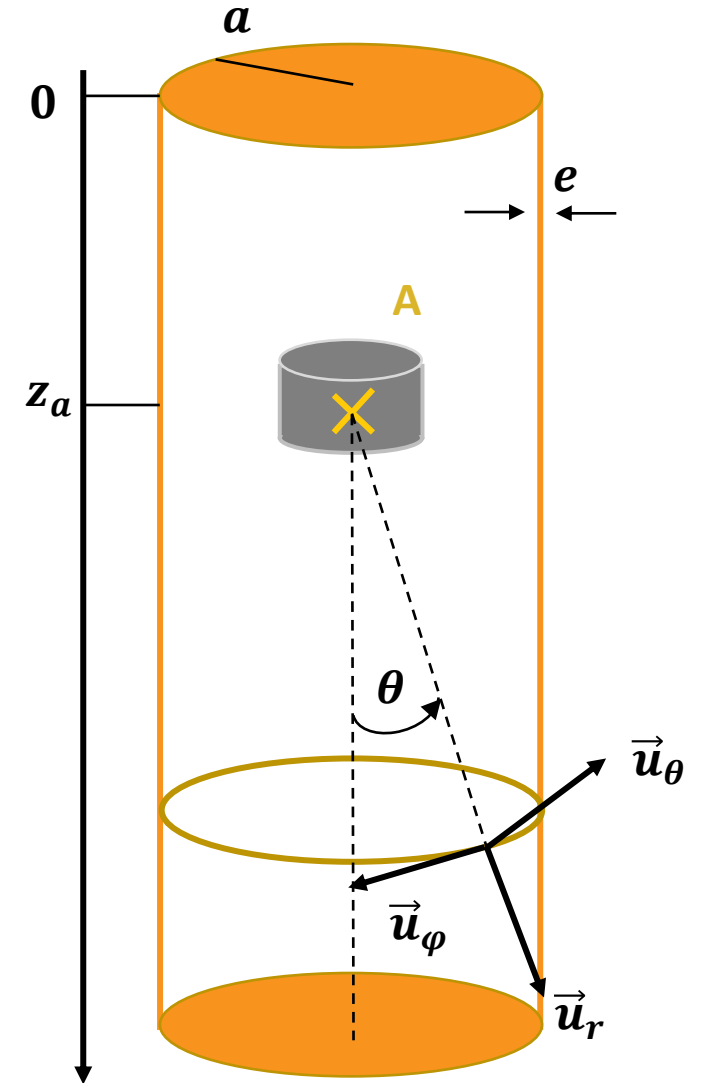
$$\overrightarrow{rot}(\vec{E} + \frac{\partial \vec{A}}{\partial t}) = \vec{0}$$

Donc il existe V (le potentiel classique) :

$$\vec{E} + \frac{\partial \vec{A}}{\partial t} = -\overrightarrow{grad} V$$

Dans la configuration : $\overrightarrow{grad} V = \vec{0}$

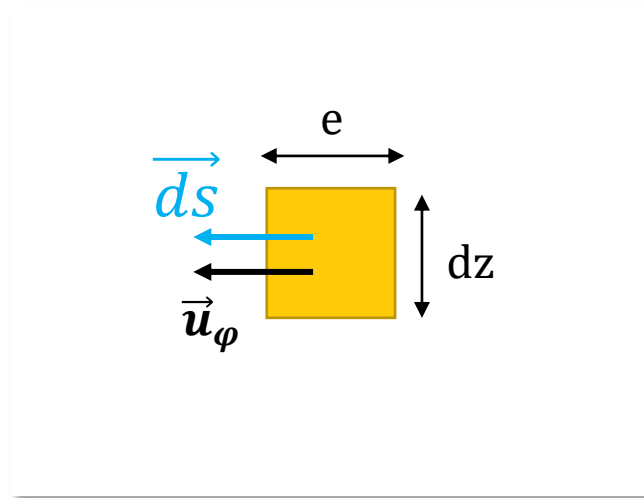
$$\vec{E} = \frac{-\partial \vec{A}}{\partial t} = \frac{-\partial \vec{A}}{\partial z_A} \frac{\partial z_A}{\partial t} = \frac{-\partial \vec{A}}{\partial z} v = -\frac{3\mu_0 M a (z - z_A) v}{4\pi (a^2 + (z - z_A)^2)^{5/2}} \vec{u}_\phi$$



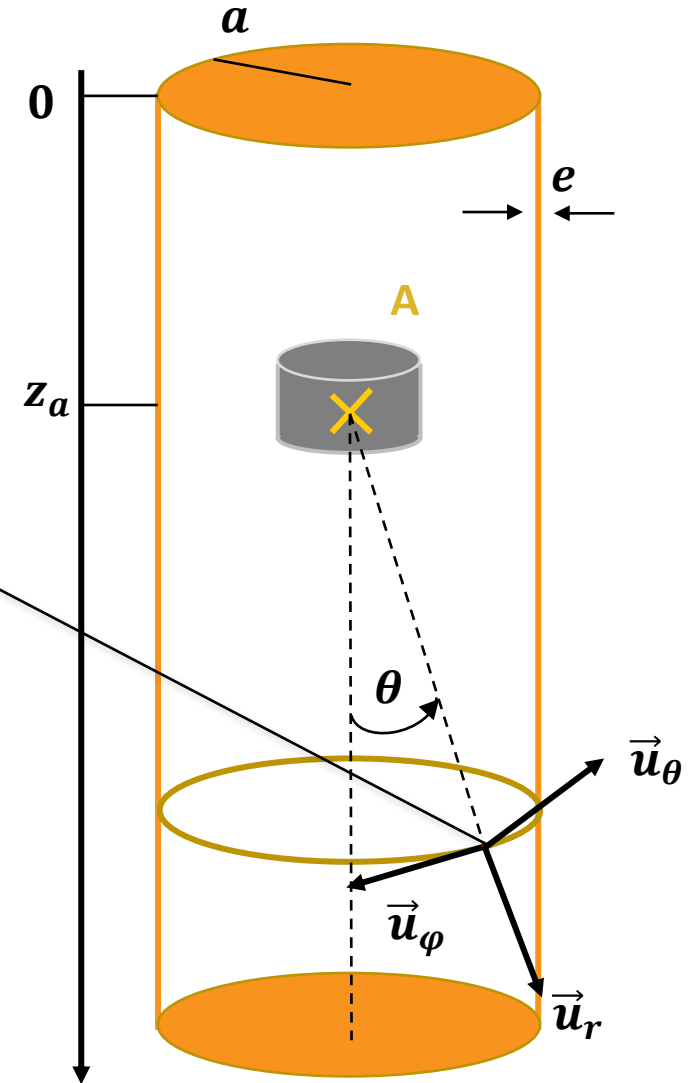
- Gardons la configuration sur la droite

e) Courant à l'intérieur d'un élément de surface de la paroi du tube

Expression de de l'intensité au travers d'un élément de surface du tube :



$$di = \vec{j} \cdot \vec{dS} = \gamma \vec{E} \cdot \vec{dS} = \gamma E \vec{u}_\phi e dz \vec{u}_\phi = - \frac{3\gamma e \mu_0 M a (z - z_A) v}{4\pi (a^2 + (z - z_A)^2)^{5/2}} dz$$



f) Force de Laplace par l'aimant sur une spire

$\vec{B}$ d'un dipôle magnétique (cours) :

$$\vec{B} = \frac{\mu_0 M}{4\pi r^3} (2\cos(\theta)\vec{u}_r + \sin(\theta)\vec{u}_\theta)$$

Force de Laplace exercée par l'aimant sur un élément de volume du tube:

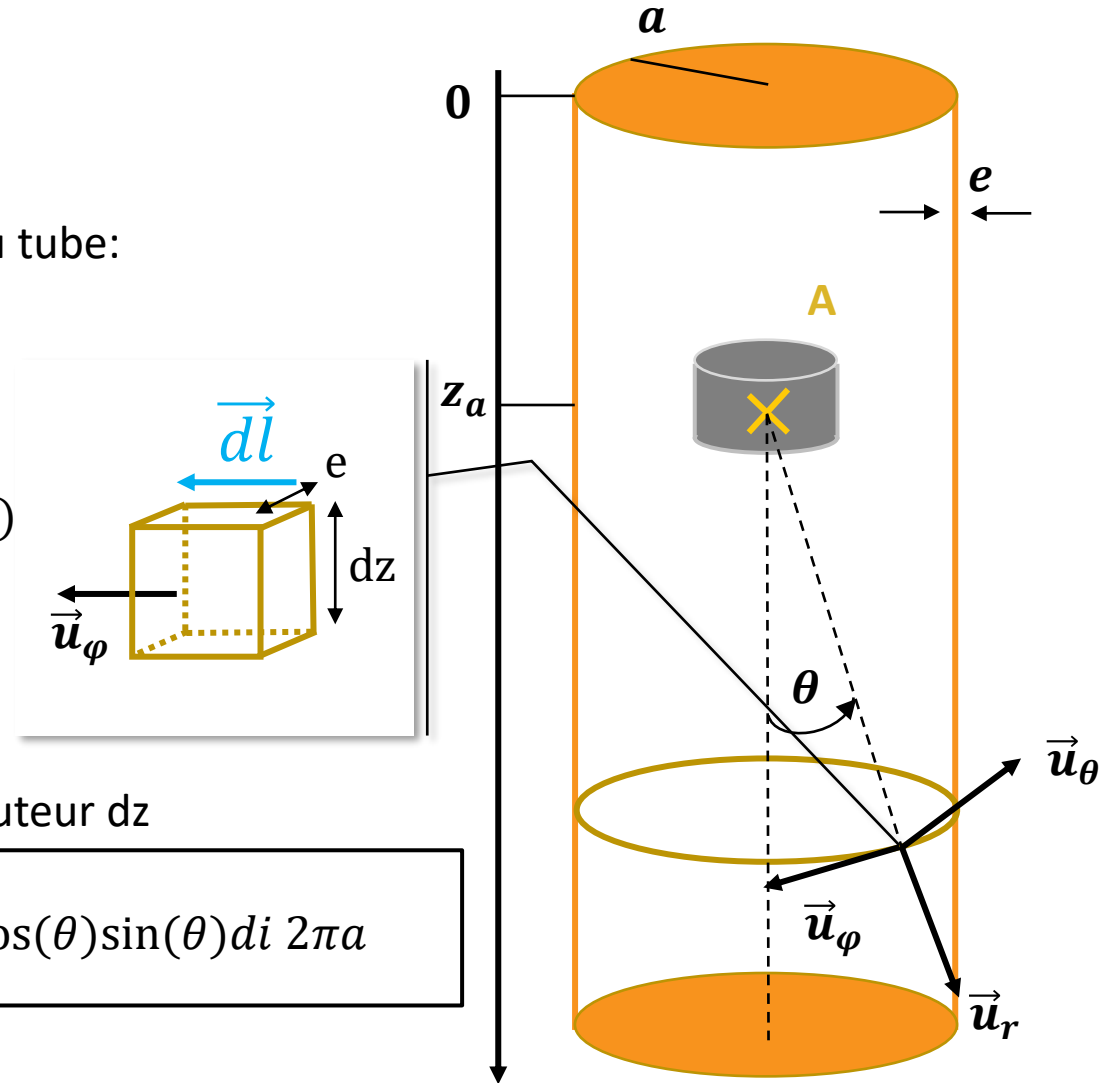
$$d^2 \vec{F} = di d\vec{l} \wedge \vec{B}$$

Or $\begin{cases} \vec{u}_\varphi \wedge \vec{u}_\theta = -\vec{u}_r \\ \vec{u}_\varphi \wedge \vec{u}_r = \vec{u}_\theta \end{cases} \rightarrow \begin{cases} d^2 F_r = -di dl B_\theta \\ d^2 F_\theta = di dl B_r \end{cases} \text{ et } \begin{cases} \vec{u}_z \cdot \vec{u}_r = \cos(\theta) \\ \vec{u}_z \cdot \vec{u}_\theta = -\sin(\theta) \end{cases}$

donc $d^2 F_z = -di dl (\cos(\theta) B_\theta + \sin(\theta) B_r)$

Force de Laplace selon $\vec{u}_z$ exercée par l'aimant sur une spire de hauteur dz

$$dF_z = \oint_C d^2 F_z = -di (\cos(\theta) B_\theta + \sin(\theta) B_r) = -\frac{3\mu_0}{4\pi r^3} m \cos(\theta) \sin(\theta) di 2\pi a$$



g) Force de Laplace par l'aimant sur l'entièreté du tube

Par des considérations géométriques :

$$dF_z = \frac{9\mu_0^2}{8\pi} M^2 a^3 \gamma e v \frac{(z - z_A)^2}{(a^2 + (z - z_A)^2)^5} dz$$

Force de Laplace selon $\vec{u}_z$ exercée par l'aimant sur tout le tube :

$$F_z = \frac{9\mu_0^2}{8\pi} M^2 a^{-5} \gamma e v \int_0^L \frac{\left(\frac{z - z_A}{a}\right)^2}{\left(1 + \left(\frac{z - z_A}{a}\right)^2\right)^5} dz$$

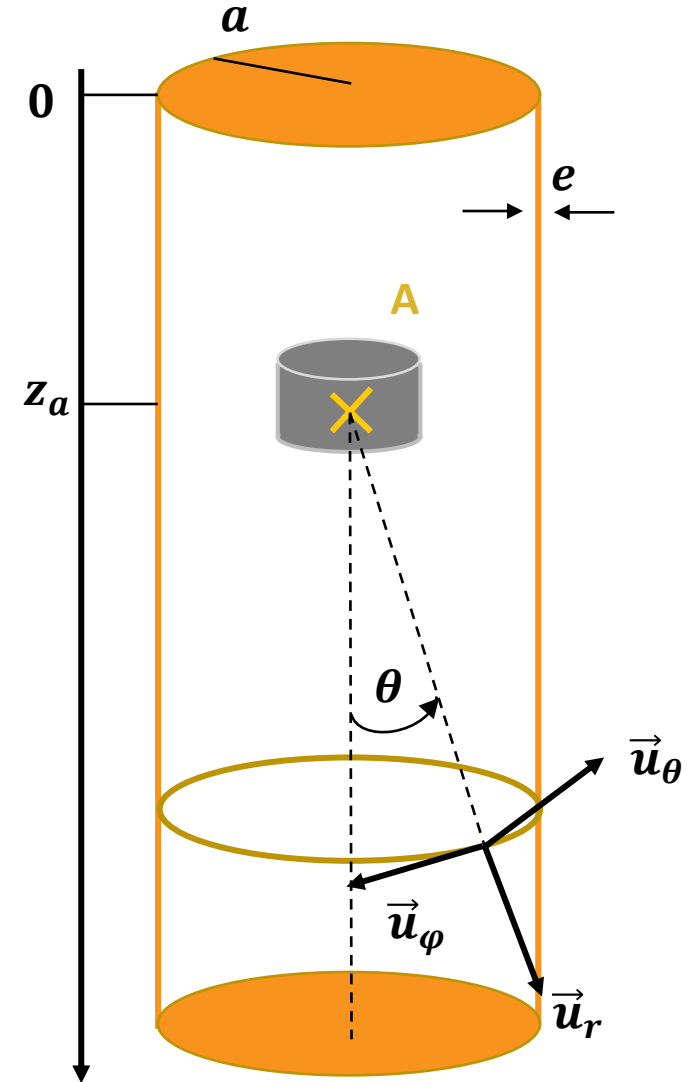
En opérant le changement de variable ($x = \frac{z - z_A}{a}$):

$$F_z = \frac{9\mu_0^2}{8\pi} M^2 a^{-4} \gamma e v \int_{\frac{z_A}{a}}^{\frac{L - z_A}{a}} \frac{x^2}{(1 + x^2)^5} dx$$

On fait l'approximation :

$$\frac{L - z_A}{a} \gg 1 \text{ et } \frac{z_A}{a} \gg 1 \text{ et } \int_{-\infty}^{+\infty} \frac{x^2}{(1 + x^2)^5} dx = \frac{5\pi}{128}$$

$$F_z = \alpha v \text{ avec } \alpha = \frac{45}{1024} \frac{M^2 \gamma e \mu_0^2}{a^4}$$



h) Principe des actions réciproques et frottement fluide

Principe des actions réciproques, la force induite par les électrons se déplaçant dans le tube sur l'aimant :

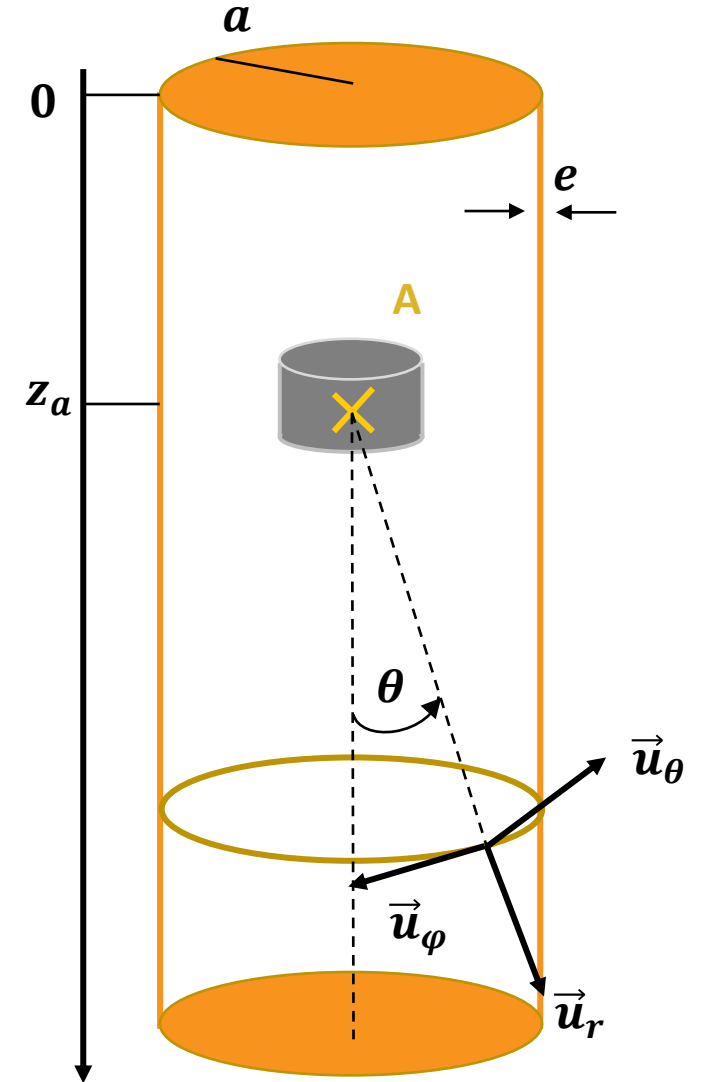
$$F'_z = -F_z = -\alpha v$$

(analogue à un frottement fluide, s'oppose à la chute)

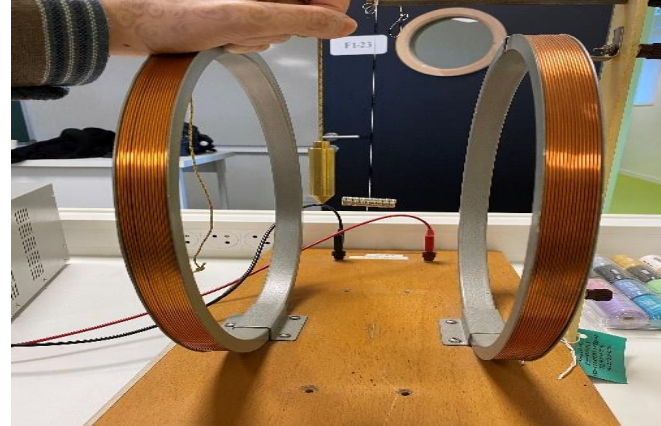
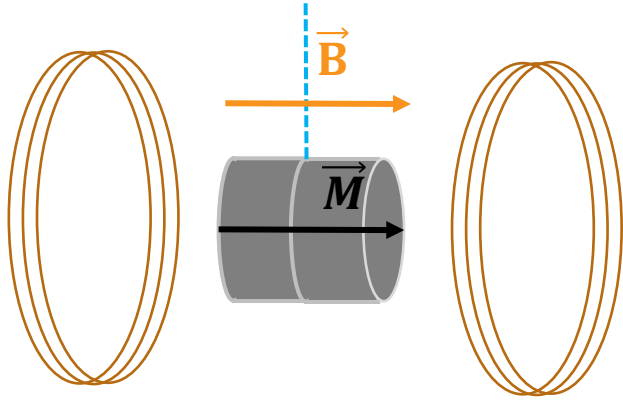
Un PFD appliqué à l'aimant donne :

$$v(t) = v_{\text{lim}} \left(1 - e^{-\frac{t}{\tau}}\right) \text{ avec } \begin{cases} v_{\text{lim}} = \frac{mg}{\alpha} \\ \tau = \frac{g}{\alpha} \end{cases}$$

$$z_a(t) = v_{\text{lim}} \left(t + \tau e^{-\frac{t}{\tau}}\right) - \tau v_{\text{lim}}$$



i) Mesure expérimentale du moment de l'aimant



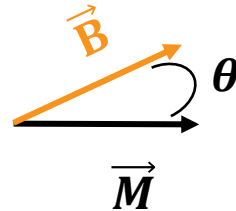
Bobine de Helmholtz : $B = 2 \text{ mT}$

Moment de $\vec{B}$ uniforme sur un dipôle magnétique :

$$\vec{\tau} = \vec{M} \wedge \vec{B}$$

TMC appliqué à l'aimant projetée selon Oz:

$$J_{Oz} \ddot{\theta} = \vec{M} \wedge \vec{B} = MB \sin(\theta)$$



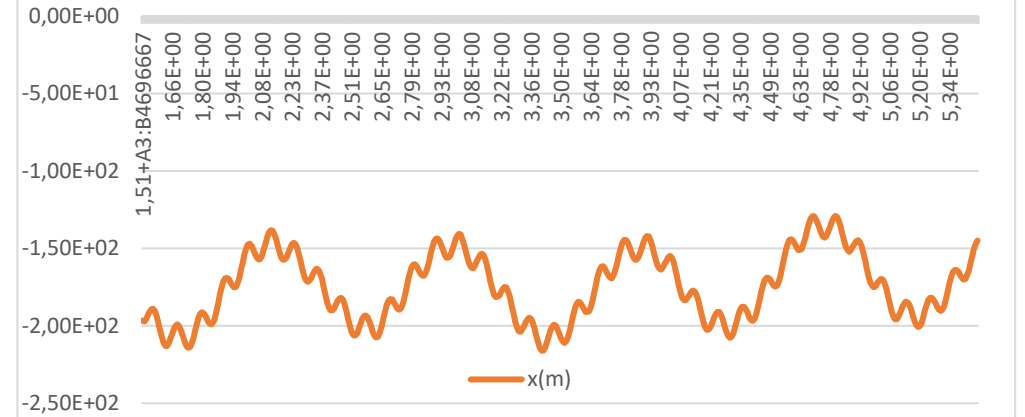
En faisant l'approximation des petites oscillations :

$$\theta = \theta_0 \cos(\omega t) \text{ avec } \omega = \frac{MB}{J_{Oz}}$$

Moment d'inertie d'un cylindre :

$$J_{Oz} = \frac{1}{4} m R^2 + \frac{1}{12} m h^2$$

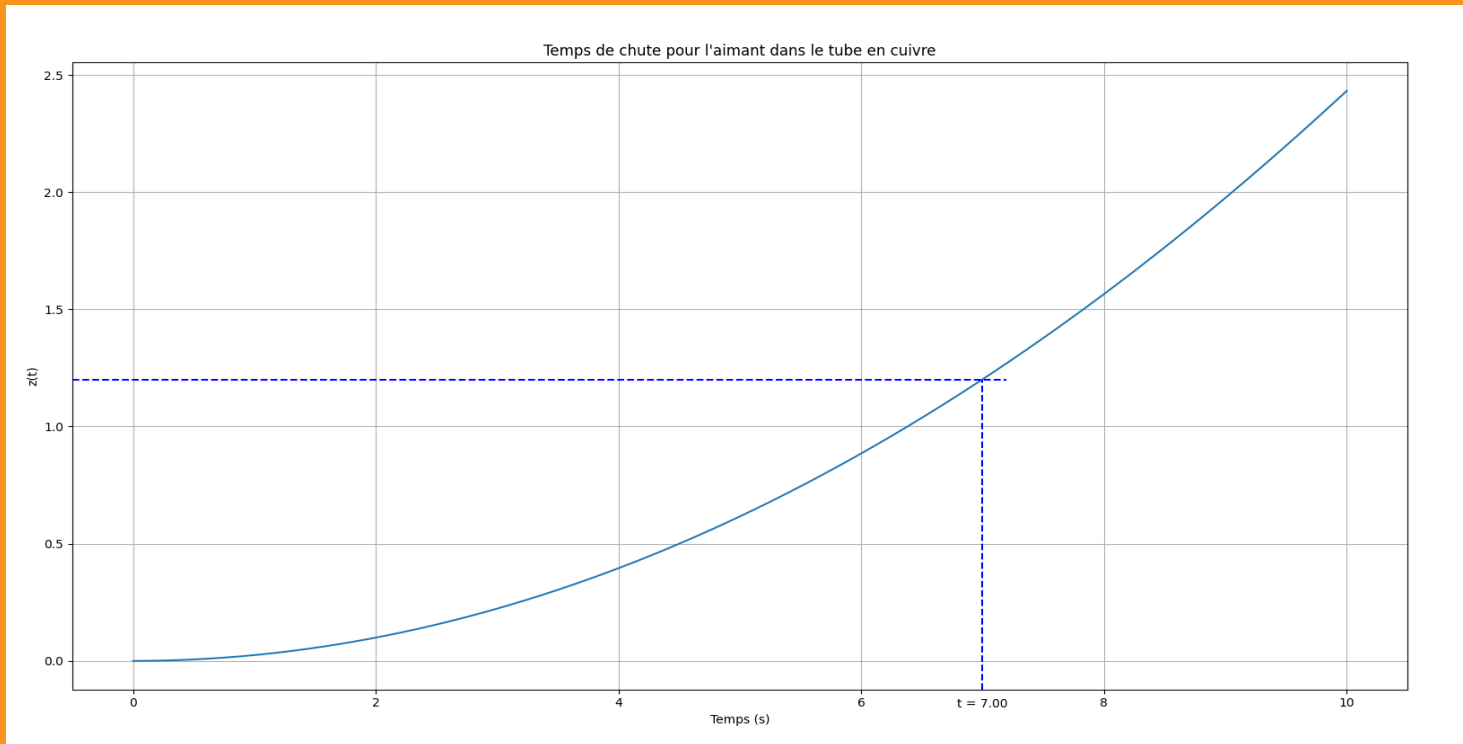
Oscillation de l'aimant au travers d'un champs B uniforme



$T = 0,1 \text{ s}$

$$M = \frac{2\pi}{BT} J_{Oz} = 1,26 \text{ A} \cdot \text{m}^{-2}$$

j) Retour à experience (temps theorique)

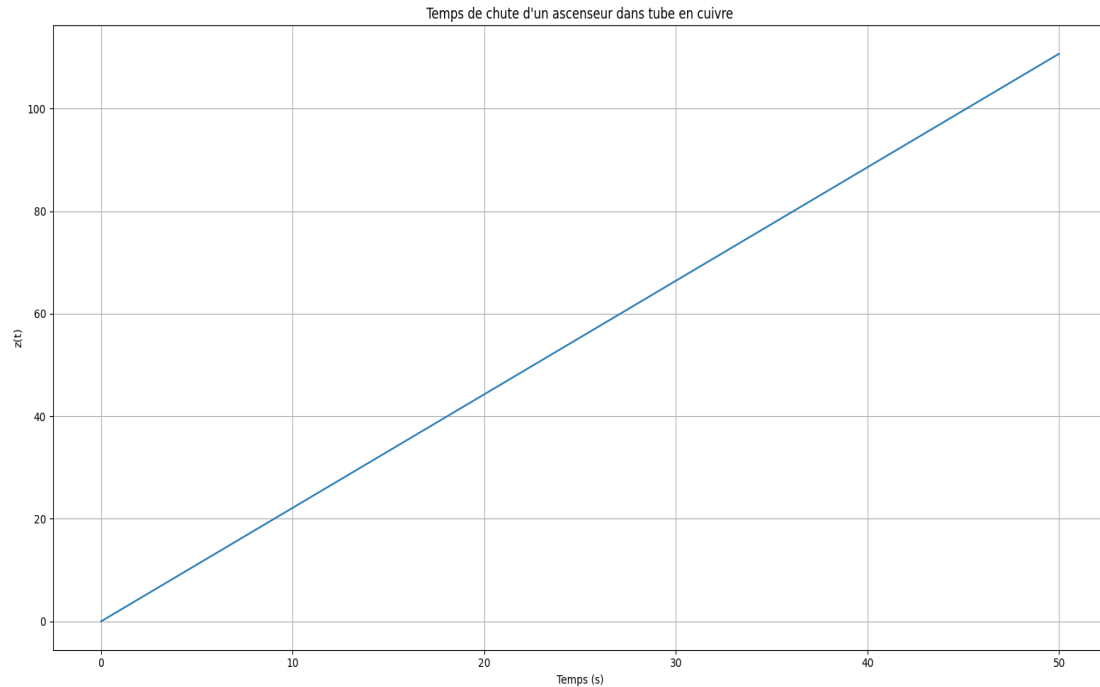


Temps de chute théorique : 7,00 s

Données

- Rayon spires : $a \approx 4 \text{ cm} = 4 * 10^{-2} \text{ m}$
- Moment magnétique de l'aimant : $M \approx 1 \text{ A.m}^{-2}$
- Epaisseur du fil : $e \approx 5 \text{ mm} = 5 * 10^{-3} \text{ m}$
- Masse de l'aimant : $m \approx 50 \text{ g} = 5 * 10^{-2} \text{ kg}$
- Conductivité du cuivre : $\gamma \approx 6 * 10^8 \text{ S.m}^{-1}$
- $\mu_0 = 4\pi * 10^{-7} \text{ H.m}^{-1}$

k) Application aux grandeurs de l'ascenseur



Données :

- Aimantation / densité volumique de moment magnétique du néodyme-fer-bore (NdFeB) :*

$$\mathcal{M} = 10^7 \text{ A.m}^{-1}$$

- Densité volumique du néodyme-fer-bore (NdFeB)*

$$\rho = 7,4 * 10^3 \text{ kg.m}^{-3}$$

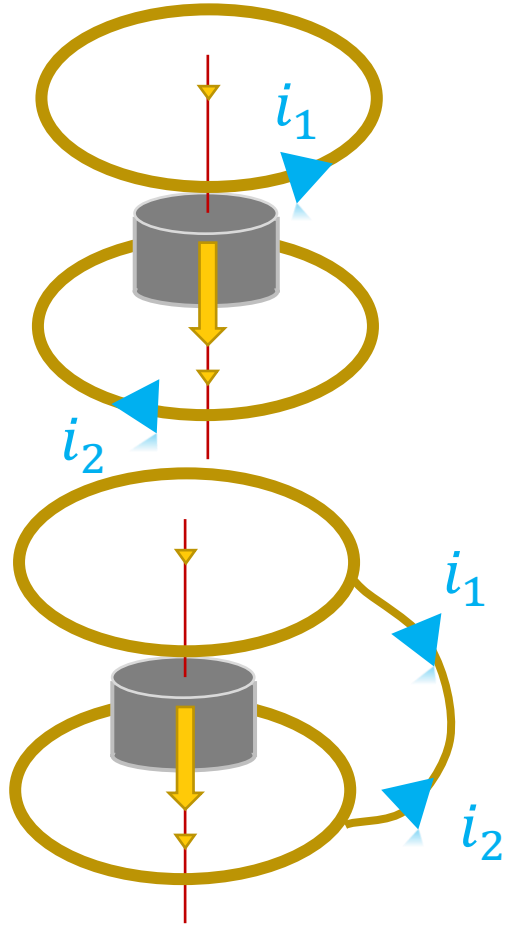
- Rayon spires : $a = 3 \text{ m}$*
- Moment magnétique de l'aimant : $M = V * \mathcal{M} = \text{A.m}^{-2}$*
- Epaisseur du tube : $e = 5 \text{ cm}$*
- Masse de l'ascenseur : $m = 300 + V * \rho$*
- Conductivité du cuivre : $\gamma \approx 6 * 10^8 \text{ S.m}^{-1}$*
- $\mu_0 = 4\pi * 10^{-7} \text{ H.m}^{-1}$*

*Pour un volume d'aimant : $V = 3 * 10^{-2} \text{ m}^3 = 30 \text{ L}$*

Vitesse moyenne : $v_{\text{moy}} = 2,25 \text{ m.s}^{-1}$ sur 100m

II – Récupération

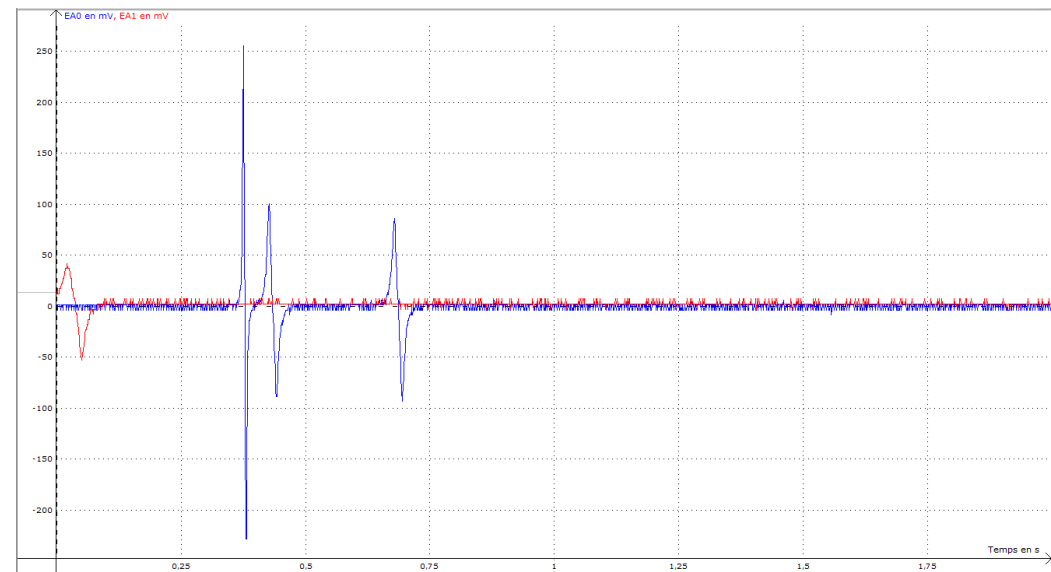
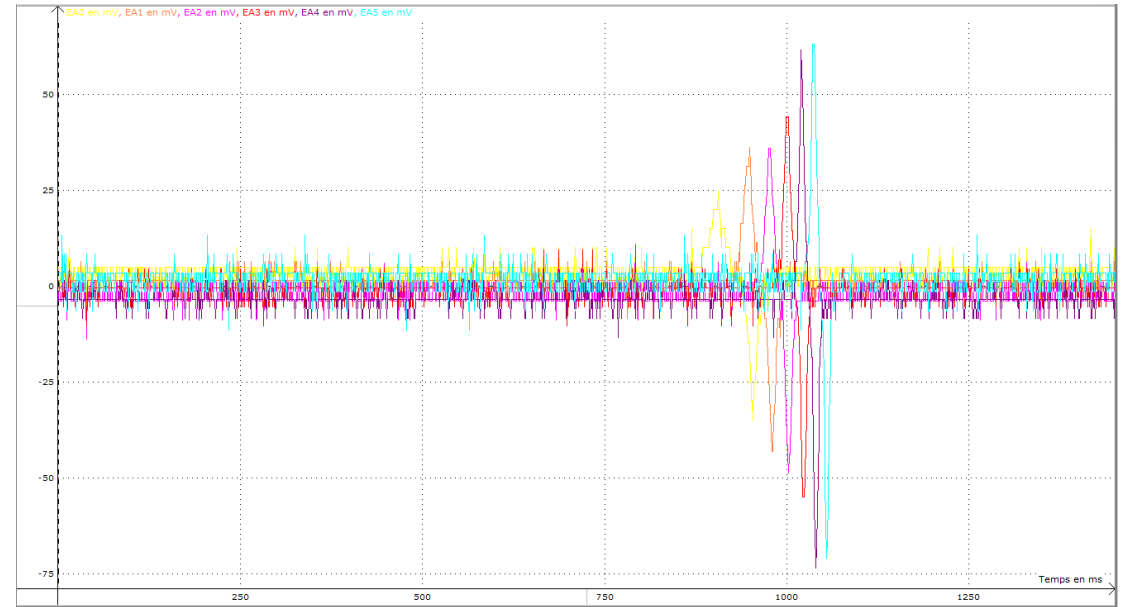
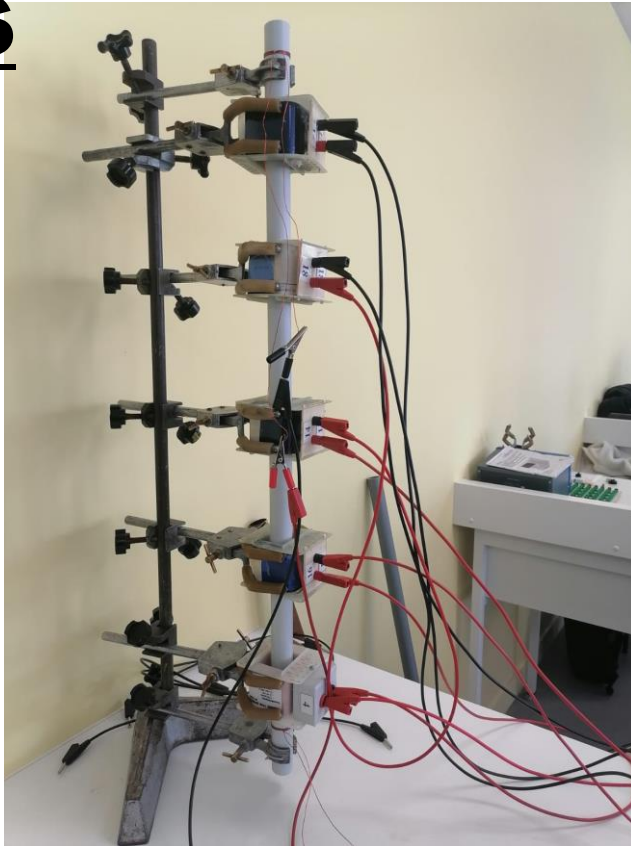
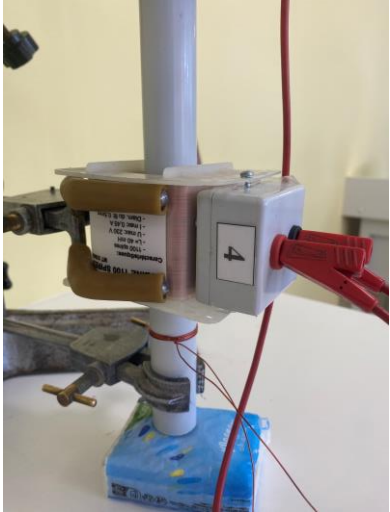
a) Première mauvaise idée (mise en série des spires)



*Tout fonctionne comme s'il n'y avait que 2 spires
Une en entrée et une autre en sortie.
On doit donc maximiser les effets de bords*

En reliant les spires, les courants se compensent.

b) Favorisation des effets de bords et temps de chutes



	Avec 5 bobines :	Sans bobine
Temps de chute (en ms)	492 +/- 1	355 +/- 1
	496 +/- 1	351 +/- 1
	486 +/- 1	354 +/- 1
	486 +/- 1	350 +/- 1

c) Stockage : condensateur chimique



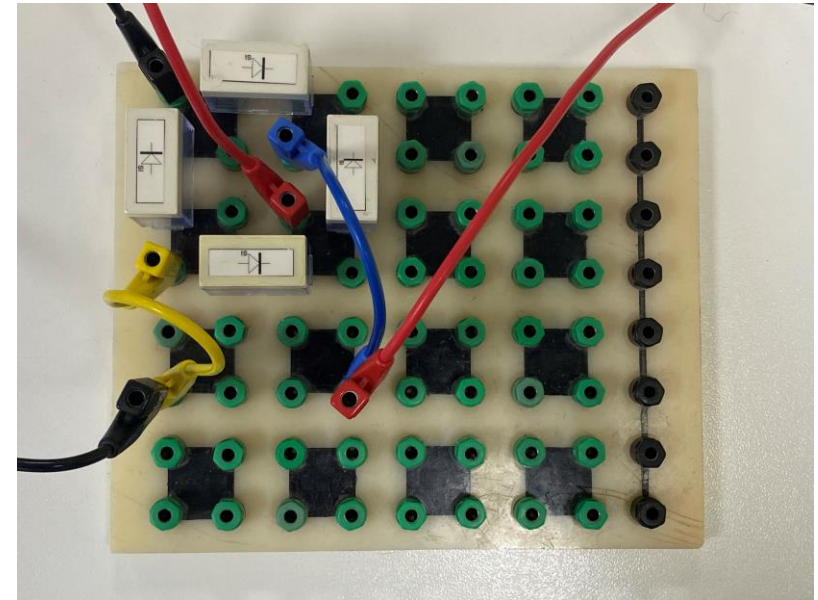
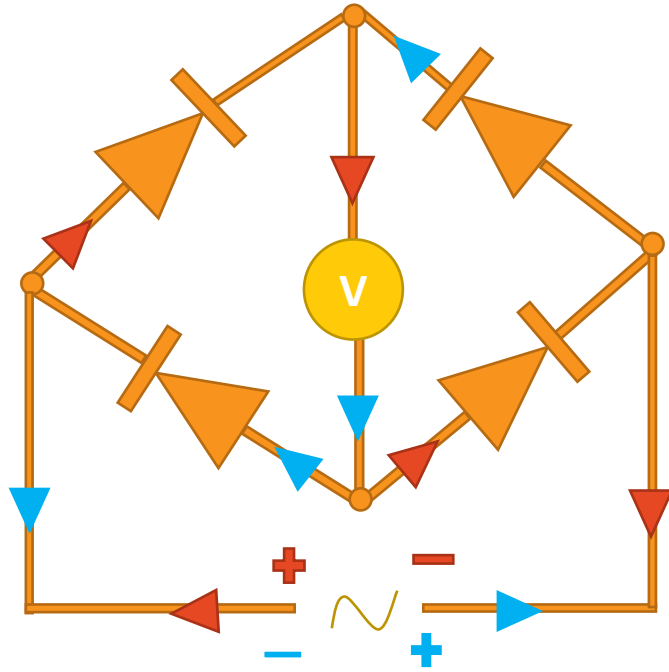
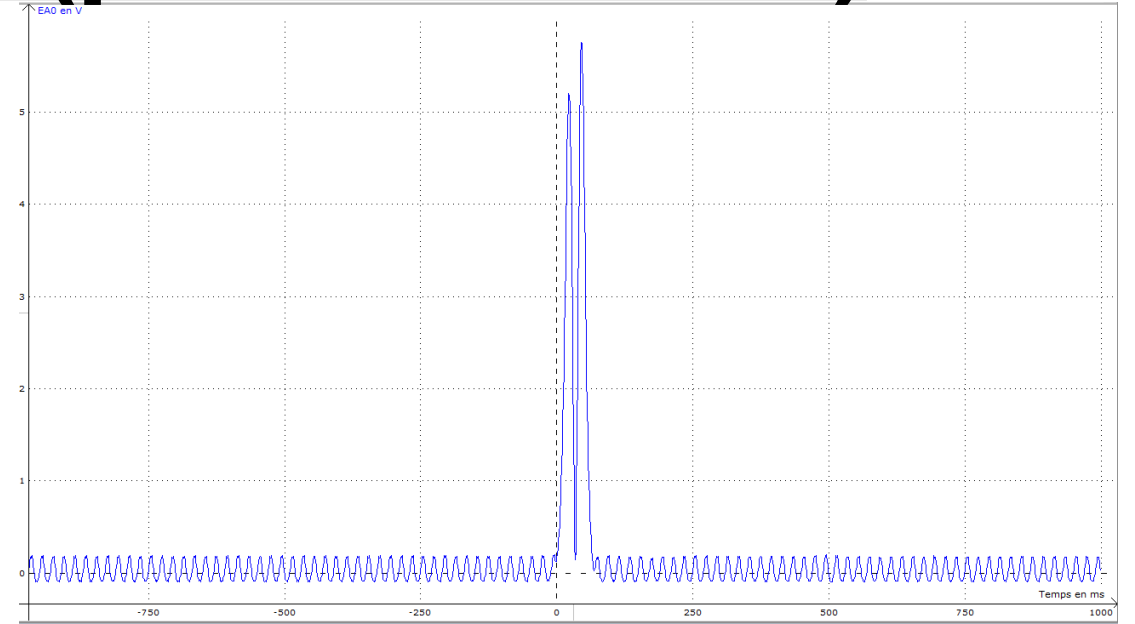
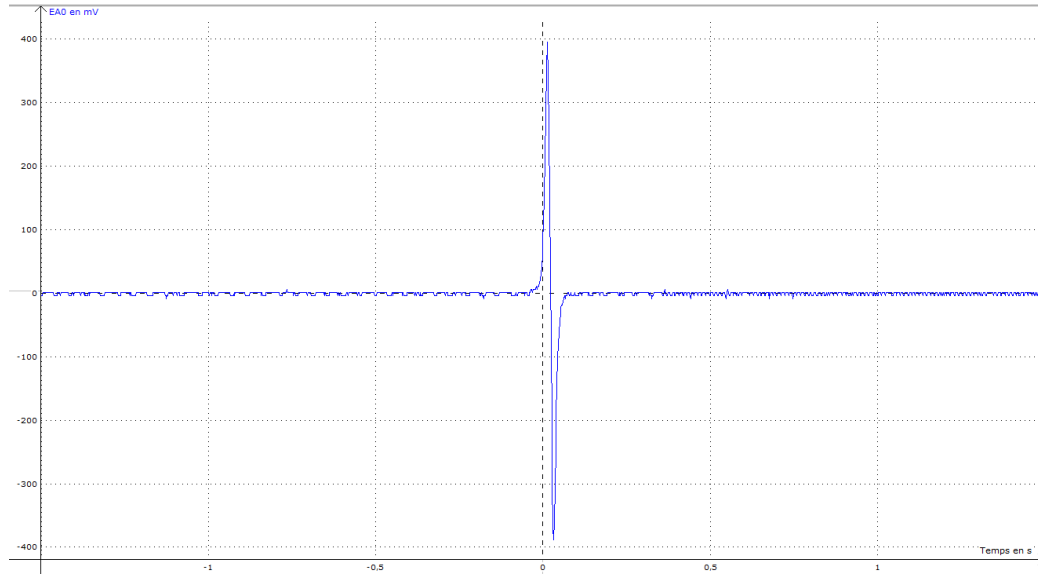
Avantages :

- Fonctionne avec de faibles courants et de faibles tensions
- Temps de réponse très court

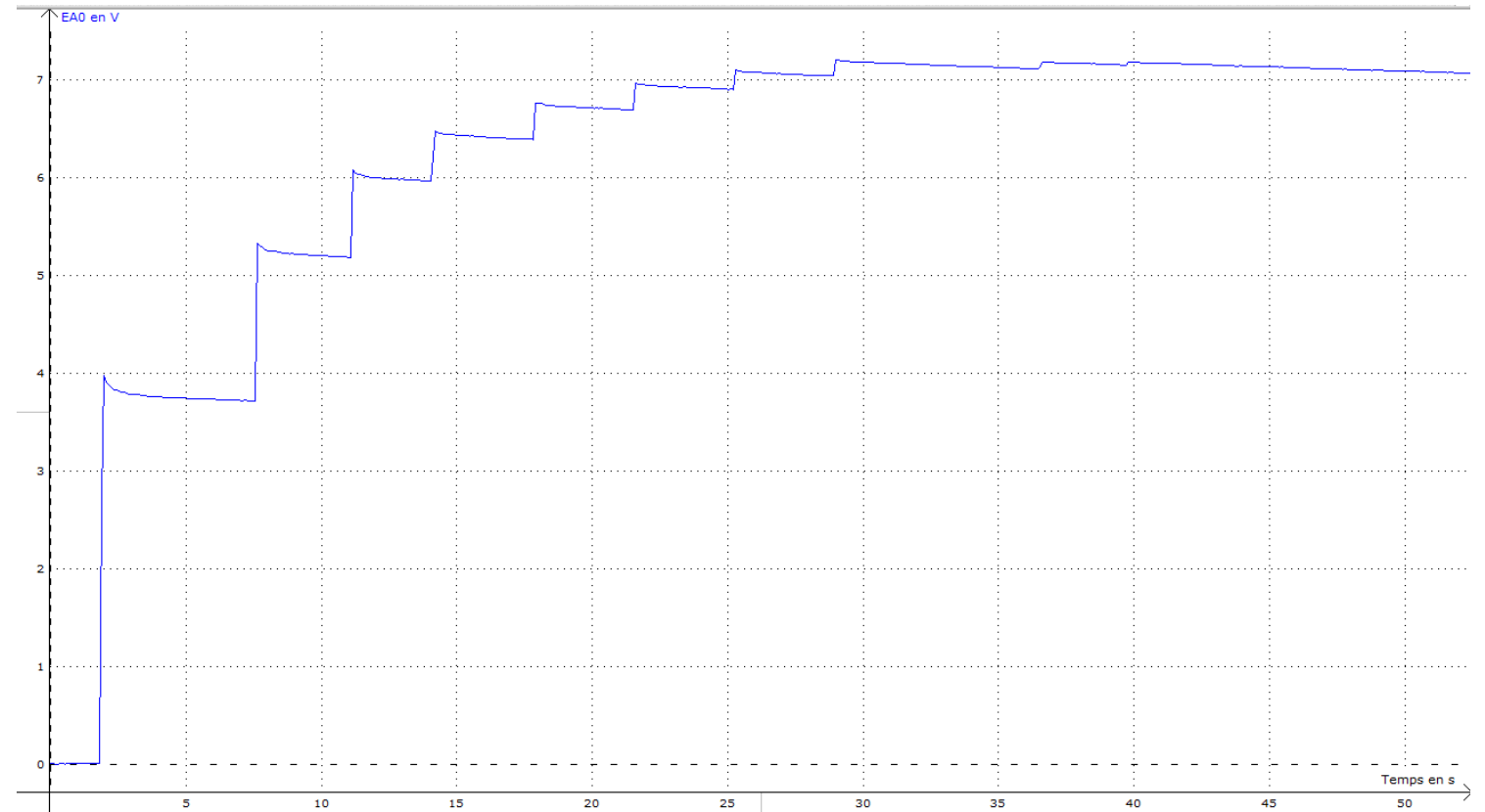
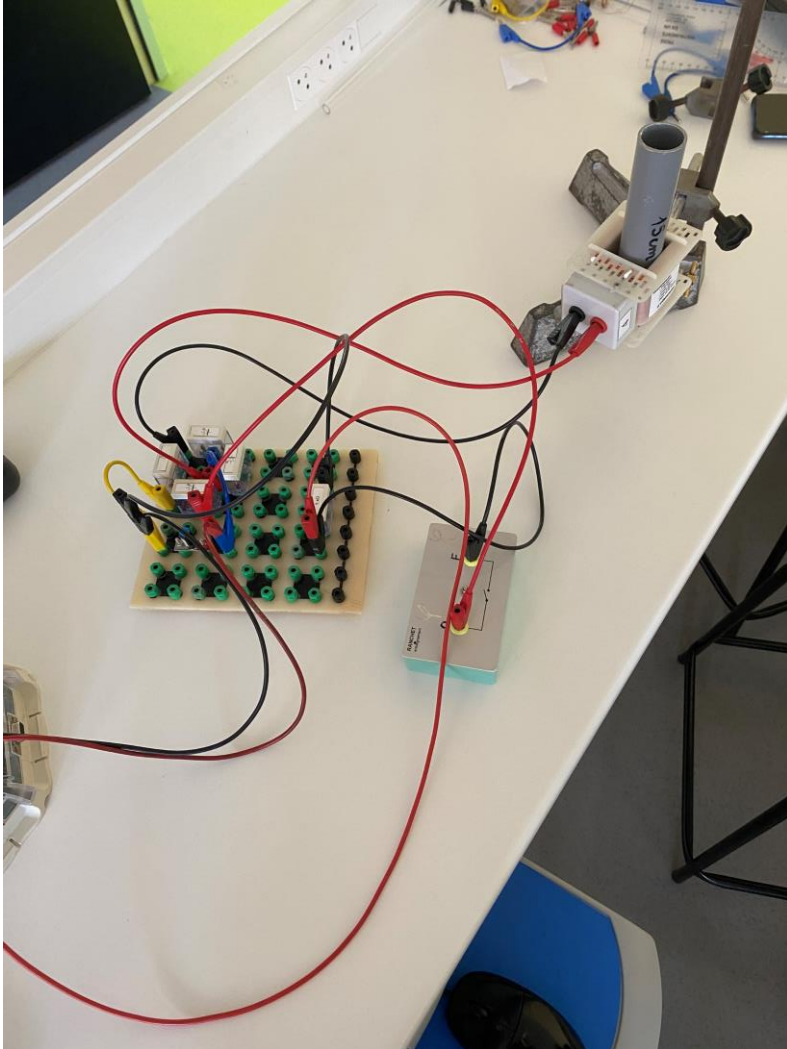
Inconvénients :

- Polarisé (explose si les bornes ne sont pas respectées)

d) Redresseur de tension (pont de diodes)



d) Récupération à la suite de 10 chutes



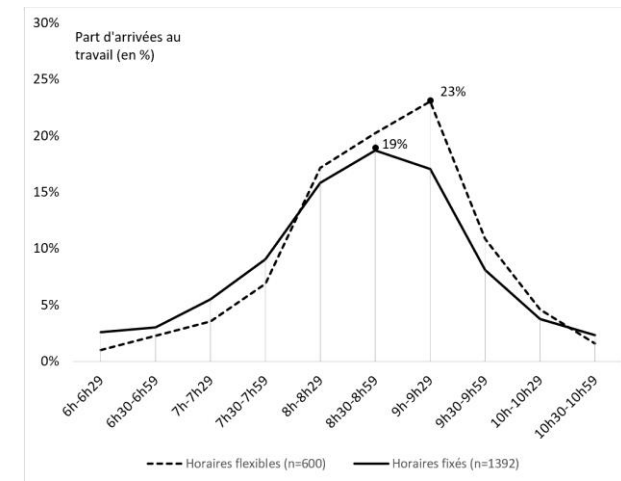
$$E_{cond} = \frac{1}{2} * C * U^2 = 8 * 10^{-3} \text{ J}$$

$$E_{chute} = mgh = 7,5 * 10^{-2} \text{ J}$$

I – Optimisation informatique (Algorithme génétique)

a) Infrastructures

Distribution selon :



Personnes dans l'ascenseur Personnes au palier

Temps de sortie

Immeuble	0	102	141	57	63	80	116	175	93	87	142
	0	79	85	105	1	107	81	190	155	67	119
	0	27	110	51	96	2	113	115	53	84	117
	0	20	123	100	96	168	38	130	81	54	128
	0	197	150	153	10	5	75	111	106	4	94
	0	0	0								0

Immeuble	0	102	141	57	63	80	116	175	93	87	142
	1	79	85	105	1	107	81	190	155	67	119
	1	27	110	51	96	2	113	115	53	84	117
	1	0	20	123	100	96	168	38	130	81	54
	0	197	150	153	10	5	75	111	106	4	94
	3	25									116

Personnes au R.D.C

Temps

Points

b) Partitions et déplacement

Définition des actions :

On associe une action à chaque chiffre compris entre 0 et 4 :

- 0 : Attendre
- 1 : Monter
- 2 : Descendre
- 3 : Faire sortir les personnes de l'ascenseur s'ils sont au R.D.C sinon attendre
- 4 : Faire rentrer les personnes de la cage d'ascenseur dans la limite de la capacité de l'ascenseur et si l'ascenseur n'est pas au R.D.C

Génération d'une partition :

Il s'agit d'une liste aléatoire de chiffre compris entre 0 et 4 de longueur la base de temps

Example :

[2, 2, 1, 2, 1, 2, 3, 1, 2, 1, 1, 0, 2, 4, 3, 3, 1, 3, 4, 1, 1, 1, 0, 0, 4, 4, 3, 2, 3, 2, 4, 1, 2, 0, 2, 2, 0, 4, 4, 3, 0, 2, 0, 3, 3, 3, 4, 0 ...]

Immeuble											
	0	102	141	57	63	80	116	175	93	87	142
	0	79	85	105	1	107	81	190	155	67	119
	0	27	110	51	96	2	113	115	53	84	117
	0	20	123	100	96	168	38	130	81	54	128
	0	197	150	153	10	5	75	111	106	4	94
0	0	0									0

Immeuble											
	1	58	97	0	149	97	49	33	126	98	88
	0	122	66	79	11	160	82	107	1	77	121
	0	120	115	106	118	74	66	91	140	98	120
	0	62	124	176	172	131	184	135	74	81	38
	0	78	150	182	50	4	107	6	111	97	19
0	0	1									0

	1	58	97	0	149	97	49	33	126	98	88
1	1	122	66	79	11	160	82	107	1	77	121
0	0	120	115	106	118	74	66	91	140	98	120
0	0	62	124	176	172	131	184	135	74	81	38
0	0	0	78	150	182	50	4	107	6	111	97
0	0	3									0

c) Evaluation et Classement

Evaluation :

Pour une configuration d'immeuble donnée, nous sommes en mesure de donner un score (arbitraire) à n'importe quelle partition :

$$\textit{Score} = \frac{(\textit{Nombre de personnes qui ont atteint le RDC})^2}{\textit{Temps moyen pour atteindre le RDC} * \textit{Coût Énergétique}}$$

Le coût énergétique est le nombre de fois où l'ascenseur est monté

Classement :

Pour un échantillon donné de partition, on utilise le tri par insertion pour classer les partitions du score le plus élevé au score le plus faible

c) Sélection et Reproduction

Sélection :

Après avoir généré un échantillon de partition, toutes les partitions sont évaluées puis classées.

On nommera :

La moitié la mieux classée des partitions : « aptes »:

L'autre moitié, les « inaptés »

Reproduction / Mutation :

À chaque partition inapte, on associe une partition apte différente.

Chaque élément de la partition inapte a une probabilité de $\frac{1}{2}$ d'être transformé en l'élément de même indice de la partition apte qui lui est associée

Exemple :

[3, 3, 1, 3, 4, 1, 1, 1, 0, 0, 4, 4, 3, ...] qui vaut 2,5 points (partition inapte)

[2, 2, 1, 2, 1, 2, 3, 1, 2, 1, 1, 0, 2, ...] qui vaut 80 points (partition apte)

La partition inapte devient :

[2,3,1,3,4,2,3,1,0,0,1,0,3...]

On concatène alors les partitions inaptées qui ont été modifiées avec les partitions aptes afin de constituer un nouvel échantillon celui de la génération 1.

On réitérera le procédé afin d'obtenir les générations suivantes dont partitions auront des scores de plus en plus élevés.

Analyse.py

```
import ascenseur as asc
import importlib
from random import *
ta_ech = 200
ta_slc = 100
nb_gen = 100

ech = [[0,asc.generation_partion(200)] for i in range(ta_ech)]

def tri_insertion(tab):
    # Parcours de 1 à la taille du tab
    for i in range(1, len(tab)):
        k = tab[i]
        j = i-1
        while j >= 0 and k[0] > tab[j][0] :
            tab[j + 1] = tab[j]
            j -= 1
        tab[j + 1] = k
```

```

def reproduction(ech):
    ech_sauf = ech[:ta_slc]
    ech_mut = ech[ta_slc:]
    for i in range(len(ech_mut)):
        for e in range(len(ech_mut[i][1])):
            if random() > 0.5:
                ech_mut[i][1][e] =
ech_sauf[i][1][e] #si les tailles n'était pas
les même
    ech = ech_sauf + ech_mut

for gen in range(nb_gen):
    for i in range(len(ech)):
        ech[i][0] =
asc.lecture_partition(ech[i][1])
        asc.reset(complet=False)
        tri_insertion(ech)
        reproduction(ech)
        asc.reset(complet=False)
print(ech[0][1])
asc.lecture_partition_aff(ech[0][1])

```

Ascenseur.py

```
from math import *
from random import *
import gui

#paramètres:
nb_et = 5 #nombre d'étages au-dessus du rez de
chaussée
lg_et = 10 #longueur d'un étage : nombre de
chambre par étage (tous les étages sont
identiques)
cap_asc = 3 #capacité de l'ascenseur

immeuble = [[-1 for i in range(lg_et)] for i in
range(nb_et)]
palier_asc = [[] for i in range((nb_et))]
asc = [nb_et,0,[]]

t = 0 #base de temps (tick), l'unité d'action
du model
```

```
#système de point
c_nrj = 1 #cout énergtique pour que l'ascenseur
monte d'un étage
c_tps = 1 #cout temporel pour que l'ascenseur
monte ou descende d'un étage

#il faut prendre en compte le temps nécessaire
moyen pour qu'un individu atteigne l'endroit
voulu, c'est à dire le bas à partir du moment
ou il prend l'ascenseur
#Rq : si on prend le temps moyen avant que
l'ascenseur vienne arrive à l'étage, on
pourrait voir apparaitre des ascenseur qui font
des navettes sans jamais atteindre le bas
#Voilà pourquoi, il faut la matrice des temps

#initialisation des variables
nrj = 0
tps = 0
nb_en_bas = 0
```

```

def repartition(immeuble, base_de_temps=200):
    print("encore")
    valeur = [4,5,6,9,16,19,17,9,6,5,4] #courbe
    en pourcentage
    l = [(i,j) for i in range(nb_et) for j in
range(lg_et)]
    shuffle(l)
    d = floor(base_de_temps/len(valeur))
    s = 0
    for m in range(len(valeur)):
        nb = floor(len(l)*valeur[m]/100)
        delta_t = [z for z in range(d*m,
d*(m+1))]
        for k in range(nb):
            i,j = l[s+k]
            immeuble[i][j] = choice(delta_t)
        s = s+nb
    for k in range(s,len(l)): #les demi-
personnes tombées à cause du floor
        i,j = l[k]
        immeuble[i][j] = choice([i for k in
range(base_de_temps)])
    return(immeuble)

```

```
def reset(complet = True):
    global immeuble
    global palier_asc
    global asc
    global t
    global nrj
    global tps
    global nb_en_bas
    if complet:
        print("reset complet")
        immeuble = [[-1 for i in range(lg_et)]]
    for i in range(nb_et):
        repartition(immeuble)
        palier_asc = [[] for i in range((nb_et))]
        asc = [nb_et, 0, []] #[etage de départ, nb de
personnes dans l'ascenseur, personnes dans
l'ascenseur]
        t = 0
        nrj = 0
        tps = 0
        nb_en_bas = 0
    reset()
```



```

def descendre(): #instruction 1
    global nrj
    if asc[0] < nb_et:
        nrj += 1
        asc[0] += 1

def monter(): #instruction 2
    if asc[0] != 0:
        asc[0] -= 1

def sortir_asc(): #instruction 3
    global tps
    global nb_en_bas
    global t
    global immeuble
    if asc[0] == nb_et:
        for (i,j) in asc[2]:
            tps += (t - immeuble[i][j]) #on
            récupère le temps qu'il a fallu à l'individu
            [i,j] pour atteindre le bas
            nb_en_bas += 1 #on ajoute un au nb
            de personnes qui ont atteint le bas
        asc[2] = [] #on vide l'ascenseur

```

```

def entrer_asc(): #on met les nb_p individus
dans l'ascenseur #instruction 4
    global palier_asc
    nb_p_asc = len(asc[2])
    place_restantes = cap_asc - nb_p_asc
    if asc[0] != nb_et and
    len(palier_asc[asc[0]]) > 0: #l'ascenseur n'es
    pas au rdc et il y a quelqu'un à sur le palier
    de l'ascenseur
        asc[2] = asc[2] +
        palier_asc[asc[0]][:place_restantes]
        palier_asc[asc[0]] =
        palier_asc[asc[0]][place_restantes:]

    #l'indiv de l'appartement (i,j) sort de chez
    lui
def sortir_ch(i,j):
    global palier_asc
    palier_asc[i].append((i,j)) #l'individu
    (i,j) est en (i,j)
    #palier_asc[0] += 1 #pour compte le nb
    d'individu sur le palier de l'asc

```

```

def descendre(): #instruction 1
    global nrj
    if asc[0] < nb_et:
        nrj += 1
        asc[0] += 1

def monter(): #instruction 2
    if asc[0] != 0:
        asc[0] -= 1

def sortir_asc(): #instruction 3
    global tps
    global nb_en_bas
    global t
    global immeuble
    if asc[0] == nb_et:
        for (i,j) in asc[2]:
            tps += (t - immeuble[i][j]) #on
            récupère le temps qu'il a fallu à l'individu
            [i,j] pour atteindre le bas
            nb_en_bas += 1 #on ajoute un au nb
            de personnes qui ont atteint le bas
        asc[2] = [] #on vide l'ascenseur

```

```

def entrer_asc(): #on met les nb_p individus
dans l'ascenseur #instruction 4
    global palier_asc
    nb_p_asc = len(asc[2])
    place_restantes = cap_asc - nb_p_asc
    if asc[0] != nb_et and
    len(palier_asc[asc[0]]) > 0: #l'ascenseur n'es
    pas au rdc et il y a quelqu'un à sur le palier
    de l'ascenseur
        asc[2] = asc[2] +
        palier_asc[asc[0]][:place_restantes]
        palier_asc[asc[0]] =
        palier_asc[asc[0]][place_restantes:]

    #l'indiv de l'appartement (i,j) sort de chez
    lui
def sortir_ch(i,j):
    global palier_asc
    palier_asc[i].append((i,j)) #l'individu
    (i,j) est en (i,j)
    #palier_asc[0] += 1 #pour compte le nb
    d'individu sur le palier de l'asc

```

```
def sortir_ch_g():  
    for i in range(nb_et):  
        for j in range(lg_et):  
            if immeuble[i][j] == t:  
                sortir_ch(i,j)
```

```
def generation_partition(longueur): #génère une  
    partition aléatoire  
    return [randint(0,4) for i in  
            range(longueur)]
```

```
def lecture_partition(p):  
    global t  
    global nrj  
    global nombre_en_bas  
    for action in p:  
        sortir_ch_g()  
        t += 1  
        if action == 1:  
            monter()  
        elif action == 2:  
            descendre()  
        elif action == 3:  
            sortir_asc()  
        elif action == 4:  
            entrer_asc()  
    return(nb_en_bas)
```